UNIVERSITAS INDONESIA

LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

SKRIPSI

MUHAMAD ABDURAHMAN
1706040095

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JULI 2021

UNIVERSITAS INDONESIA

LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Ilmu Komputer

MUHAMAD ABDURAHMAN
1706040095

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JULI 2021

HALAMAN PERSETUJUAN

Judul	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia
Nama	:	Muhamad Abdurahman
NPM	:	1706040095

Laporan Skripsi ini telah diperiksa dan disetujui.

Tanggal Bulan Tahun

Fariz Darari, Ph.D.

Pembimbing Skripsi

HALAMAN PERNYATAAN ORISINALITAS

Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.

Nama	:	**Muhamad Abdurahman**
NPM	:	**1706040095**
Tanda Tangan	:	
Tanggal	:	**Tanggal Bulan Tahun**

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh	:	
Nama	:	Muhamad Abdurahman
NPM	:	1706040095
Program Studi	:	Ilmu Komputer
Judul Skripsi	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Ilmu Komputer pada Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing 1	:	Fariz Darari, Ph.D.	(	)
Penguji 1	:	Penguji Pertama Anda	(	)
Penguji 2	:	Penguji Kedua Anda	(	)

Ditetapkan di	:	Depok
Tanggal	:	Tanggal Bulan Tahun

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama	:	Muhamad Abdurahman
NPM	:	1706040095
Program Studi	:	Ilmu Komputer
Fakultas	:	Ilmu Komputer
Jenis Karya	:	Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif (Non-exclusive Royalty Free Right)** atas karya ilmiah saya yang berjudul:

Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (database), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyatan ini saya buat dengan sebenarnya.

Dibuat di	:	Depok
Pada tanggal	:	Tanggal Bulan Tahun

Yang menyatakan

(Muhamad Abdurahman)

KATA PENGANTAR

Lorem Ipsum

Depok, Tanggal Bulan Tahun

Muhamad Abdurahman

ABSTRAK

Nama	:	Muhamad Abdurahman
Program Studi	:	Ilmu Komputer
Judul	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

Peraturan perundang-undangan pada umumnya tersedia dalam bentuk berkas PDF yang bersifat tidak *machine-readable*. *Knowledge graph* adalah struktur data berbentuk *graph* yang menggambarkan entitas dunia nyata beserta keterkaitannya. Pada penelitian ini berfokus pada pengembangan *framework* untuk mengonversi peraturan perundang-undangan di Indonesia dari dokumen PDF menjadi *knowledge graph* dan memberikan contoh *use case* dari *knowledge graph* tersebut. *Knowledge graph* peraturan perundang-undangan tersebut mengandung data-data berupa metadata peraturan, struktur peraturan, teks peraturan, serta relasi antara peraturan seperti amandemen dan rujukan. Dari 1669 dokumen PDF yang diunduh dari situs Jaringan Dokumentasi dan Informasi Hukum DPR RI, 784 dokumen diantaranya berhasil dikonversi menjadi *knowledge graph* dengan total 1.159.231 *triple*. Salah satu peraturan yang berhasil dikonversi adalah UU 11/2020 tentang Cipta Kerja yang kontennya bersifat relatif kompleks dan berukuran besar. Penelitian ini juga mendemonstrasikan pemanfaatan *knowledge graph* tersebut untuk *chat bot*, *visualisasi*, dan *query*.

Kata kunci:
Knowledge Graph, Peraturan perundang-undangan, SPARQL, *Resource Definition Framework*

ABSTRACT

Name : Muhamad Abdurahman
Program : Ilmu Komputer
Title : Lex2KG: Automated Legal KG Population from Indonesian Legal Documents

//TODO: Ini ntar aja klo yg abstrak bhs indo udah final

Key words:
Keyword one, keyword two

DAFTAR ISI

HALAMAN JUDUL i

LEMBAR PERSETUJUAN ii

LEMBAR PERNYATAAN ORISINALITAS iii

LEMBAR PENGESAHAN iv

LEMBAR PERSETUJUAN PUBLIKASI ILMIAH v

KATA PENGANTAR vi

ABSTRAK vii

Daftar Isi ix

Daftar Gambar xi

Daftar Tabel xii

Daftar Kode Program xiii

Daftar Lampiran xiii

1 PENDAHULUAN 1
- 1.1 Latar Belakang 1
- 1.2 Permasalahan 4
- 1.3 Batasan Permasalahan 5
- 1.4 Metodologi Penelitian 5
- 1.5 Sistematika Penulisan 6

2 TINJAUAN PUSTAKA 7
- 2.1 Knowledge Graph 7
- 2.2 Resource Description Framework 8
- 2.3 SPARQL 9
  - 2.3.1 Contoh Query 1 9
  - 2.3.2 Contoh Query 2 10
  - 2.3.3 Contoh Query 3 10
- 2.4 Peraturan Perundang-undangan 11
- 2.5 *European Legislation Identifier* (ELI) 13
- 2.6 Schema.org 13
- 2.7 Wikidata 14

3 PERANCANGAN **15**
3.1 *Competency Questions* 15
3.2 Perancangan URI 16
3.3 Perancangan URI Class 17
3.4 Perancangan URI Komponen Peraturan Perundang-undangan 17
3.5 Perancangan URI Properti 19

4 IMPLEMENTASI **22**
4.1 OCR Ulang Berkas PDF 22
4.2 Ekstraksi Berkas PDF menjadi Data *Span* 24
4.3 Membersihkan *Noise* dari Halaman 25
4.4 Penggabungan Data Berkas PDF Asli dan Hasil OCR Ulang 26
4.5 Pengelompokan *span* menjadi Komponen 27
4.5.1 Komponen Terurut 29
4.5.2 Komponen Amandemen 29
4.6 Deteksi Sitasi 31
4.7 Konstruksi KG 33
4.8 Query 33

5 EVALUASI **35**
5.1 *Use Case Evaluation* 35
5.1.1 SPARQL Query 35
5.1.2 Visualisasi KG 36
5.1.3 *Question Answering Chatbot* 37
5.2 *Large Scale Evaluation* 37

6 PENUTUP **38**
6.1 Kesimpulan 38
6.2 Saran 38

Daftar Referensi **39**

DAFTAR GAMBAR

1.1 Cuplikan ekstraksi data terstruktur dan data tidak terstruktur dari dokumen PDF UU 11/2020 tentang Cipta Kerja 2
1.2 Bagan Alir Pembuatan dan Query KG Peraturan Perundang-undangan . . 3
1.3 Metodologi Penelitian 6

2.1 Contoh *Knowledge Graph*. Sumber: `https://yashuseth.files.wordpress.com/2019/10/knowledge-graph.jpg` 7
2.2 Contoh Triple 8

4.1 Contoh PDF yang akan diekstraksi 24
4.2 Contoh header 25
4.3 Contoh nomor tidak terpindai 26
4.4 Ilustrasi Ekstraksi Daftar Span menjadi Data Struktur Komponen 28
4.5 Contoh Amendemen pada UU Nomor 11 Tahun 2020 30
4.6 Deteksi Komponen Amandemen 31
4.7 Konstruksi KG 33

5.1 Output VizKG untuk Kode 5.2 37

DAFTAR TABEL

2.1 Output Query 1 10
2.2 Output Query 3 10
2.3 Output Query 3 11

3.1 Deskripsi Class Lex2KG 17
3.2 Konstruksi URI untuk Peraturan Perundang-undangan 18
3.3 Konstruksi URI untuk komponen Peraturan Perundang-undangan 19
3.4 Nama, domain, dan range properti Lex2KG 21

4.1 Metadata PDF Peraturan Perundang-undangan 22
4.2 Long table caption. 32
4.3 Contoh teks dan URI yang terdeteksi dari teks tersebut 33

5.1 Output Query Kode 5.1 36

DAFTAR KODE PROGRAM

1.1 Contoh KG tentang metadata untuk suatu dokumen legal dengan *vocabulary* Schema.org . . . 4
2.1 Contoh penulisan sebuah *triple* dalam turtle . . . 8
2.2 Contoh penulisan lebih singkat sebuah *triple* dalam turtle . . . 9
2.3 Query 1 . . . 10
2.4 Query 2 . . . 10
2.5 Query 3 . . . 11
4.1 Command untuk melakukan instalasi program OCR pada Linux OS . . . 23
4.2 Command untuk menjalankan OCR . . . 23
4.3 Konfigurasi Tesseract OCR . . . 23
4.4 Hasil ekstraksi Gambar 4.1 . . . 25
4.5 Data jika nomor 9 tidak berhasil terpindai . . . 27
4.6 Data jika nomor 9 terpindai . . . 27
4.7 Contoh teks pada Undang-Undang Nomor 13 Tahun 2003 Pasal 158 Ayat (4) beserta sitasi yang terdeteksi . . . 32
4.8 Command untuk melakukan instalasi Fuseki . . . 34
4.9 Command untuk menjalankan Fuseki dengan berkas KG lex2kg.ttl . . . 34
5.1 SPARQL query untuk “Tampilkan semua peraturan serta peraturan yang ditimbangnya.” . . . 35
5.2 Visualisasi “Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya” menggunakan VizKG . . . 36
5.3 Skenario 1 Chatbot . . . 37

DAFTAR LAMPIRAN

Lampiran 1. CHANGELOG 41

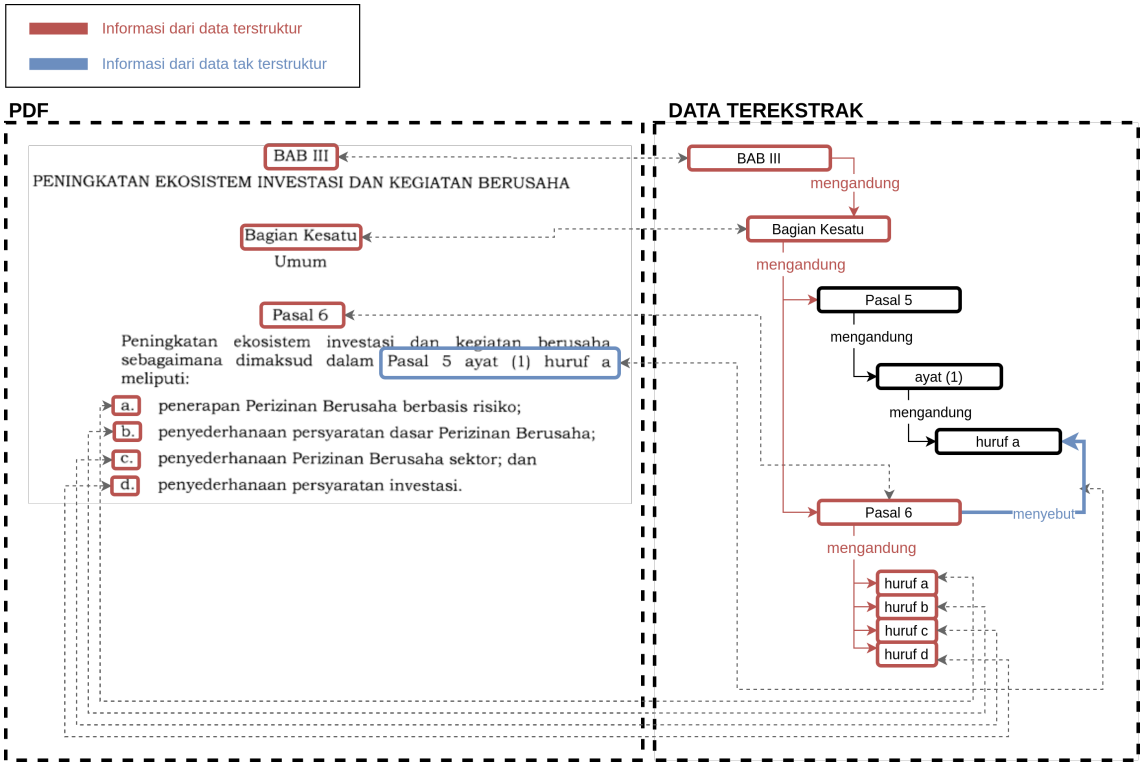
BAB 1
PENDAHULUAN

1.1 Latar Belakang

Peraturan perundang-undangan adalah peraturan tertulis yang memuat norma hukum yang mengikat secara umum dan dibentuk atau ditetapkan oleh lembaga negara atau pejabat yang berwenang melalui prosedur yang ditetapkan dalam peraturan perundang-undangan, sesuai yang dijelaskan dalam Undang-Undang Republik Indonesia Nomor 12 Tahun 2011 (selanjutnya disebut dengan UU 12/2011). Peraturan perundang-undangan dapat digunakan untuk menjawab pertanyaan-pertanyaan yang berkaitan dengan hukum, seperti:

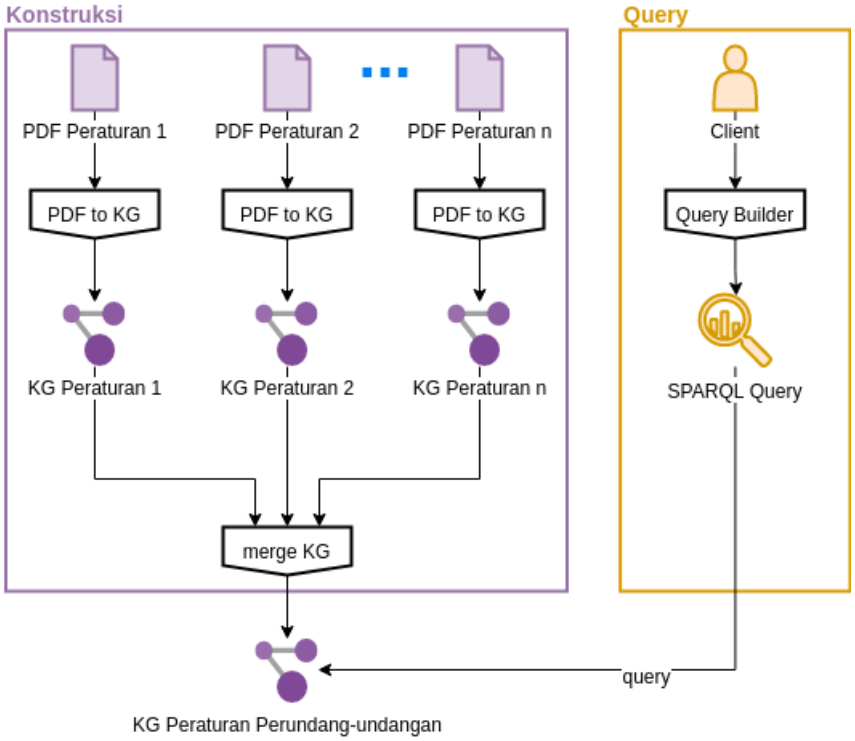
- Peraturan apa yang berlaku pada daerah X yang disahkan di tahun Y?
- Apa relasi suatu peraturan dengan peraturan lain?
- Mana saja peraturan yang membahas tentang topik W atau X tapi bukan topik Y dan bukan Z?
- Apa isi dari Pasal X ayat (Y) huruf Z dari suatu peraturan perundang-undangan?
- Apa saja bab-bab yang terdapat pada peraturan X?

Pertanyaan-pertanyaan tersebut umumnya biasa dijawab oleh seorang ahli hukum, yang hanya dapat dilakukan dalam skala kecil dan biaya relatif mahal. Komputer dapat menjadi alternatif untuk aplikasi tersebut dalam skala lebih besar dan biaya lebih murah, jika peraturan perundang-undangan berupa data terstruktur yang dapat diolah oleh komputer. Akan tetapi, data peraturan perundang-undangan umumnya dibuat dalam bentuk semi-terstruktur, yaitu berupa dokumen PDF dengan struktur konten seperti aturan penomoran dan aturan pembagian (*sectioning*), sementara penulisan isi peraturannya sendiri dalam bentuk teks bahasa manusia (natural language) yang tak terstruktur (unstructured).



Gambar 1.1: Cuplikan ekstraksi data terstruktur dan data tidak terstruktur dari dokumen PDF UU 11/2020 tentang Cipta Kerja

Gambar 1.1 menampilkan contoh ekstraksi informasi dari UU 11/2020 tentang Cipta Kerja. Pada gambar tersebut terlihat bahwa terdapat dua jenis ekstraksi data yaitu ekstraksi dari data terstruktur (merah) dan ekstraksi dari data tidak terstruktur (biru). Contoh data terstruktur pada dokumen tersebut adalah *section* pada dokumen seperti **Bab III**, **Bagian Kesatu**, **Pasal 6**, dan **huruf a - d**. Contoh data tidak terstruktur pada dokumen tersebut adalah semua teks yang terdapat pada *section* tersebut seperti teks "penerapan Perizinan Berusaha berbasis risiko;" yang terdapat pada **huruf a**. Manusia perlu membaca manual untuk mengetahui bahwa UU 11/2020 mengandung **Bab III**, **Bab III** mengandung **Bagian Kesatu**, **Bagian Kesatu** mengandung **Pasal 6**, dan **Pasal 6** mengandung **huruf a - d**. Pada gambar tersebut, dalam mengekstraksi informasi "UU 11/2020 Pasal 6 merujuk kepada Pasal 5 ayat (1) huruf a", manusia pertama-tama perlu membaca dari awal teks pasal-pasal pada UU perujukan (citation), dan kemudian meng-*infer* informasi lengkap berdasarkan konteks peraturan perundang-undangan.



Gambar 1.2: Bagan Alir Pembuatan dan Query KG Peraturan Perundang-undangan

Knowledge graph (KG) adalah struktur data yang (i) menggambarkan entitas dunia nyata dan keterkaitannya, tersusun dalam bentuk *graph*, (ii) mendefinisikan kelas entitas dan relasi antar entitas dalam bentuk skema, (iii) membolehkan sembarang entitas memiliki relasi satu sama lain dan (iv) mencakup berbagai topik domain (Paulheim, 2016). Dengan melakukan ekstraksi data dari dokumen yang bersifat semi-terstruktur, peraturan perundang-undangan dapat diubah menjadi KG. Sebagai manfaatnya, dapat fitur-fitur berbasis *query* seperti *question answering*, *reasoning* dan visualisasi data. Gambar 1.2 menampilkan proses konstruksi KG dari beberapa berkas PDF peraturan perundang-undangan dan *query* pada KG tersebut. Pada skripsi ini, akan dilakukan konversi dokumen peraturan perundang-undangan menjadi data terstruktur dalam bentuk KG, dan memberikan contoh aplikasi dari KG peraturan perundang-undangan tersebut. Beberapa usaha telah dilakukan untuk membuat *vocabulary* untuk KG peraturan perundang-undangan seperti ELI [1], Schema.org [2], dan Wikidata [3].

[1] `https://eur-lex.europa.eu/eli-register/about.html`
[2] `https://schema.org`
[3] `https://www.wikidata.org`

```
@prefix schema: <http://schema.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<http://data.europa.eu/eli/dir/1980/181/oj>
  a schema:Legislation ;
  schema:about <http://eurovoc.europa.eu/1896> ;
  schema:isBasedOn <http://njh.me/11957E100.html> ;
  schema:legislationDate "1979-12-20"^^schema:Date ;
  schema:legislationPassedBy "Council of the European Union"^^xsd:string ;
  schema:name "Council Directive 80/181/EEC of 20 December 1979 on the approximation
  ↪ of the laws of the Member States relating to units of measurement and on the
  ↪ repeal of Directive 71/354/EEC"^^xsd:string ;

<http://eurovoc.europa.eu/1896>
  a schema:Thing ;
  schema:name "Metrology"^^xsd:string .

<http://njh.me/11957E100.html> a schema:Legislation .
```

Kode 1.1: Contoh KG tentang metadata untuk suatu dokumen legal dengan *vocabulary* Schema.org

Pada Kode 1.1 diberikan contoh KG yang dibuat dengan *vocabulary* Schema.org. Contoh KG tersebut memuat metadata peraturan seperti tentang apa peraturan tersebut (`schema:about`), apa dasar dari peraturan tersebut (`schema:isBasedOn`), dan nama dari peraturan tersebut (`schema:name`). Akan tetapi, baik Schema.org maupun vocabulary-vocabulary lainnya seperti pada ELI dan Wikidata, tidak dapat merepresentasikan struktur beserta isi peraturan. Selain itu, *vocabulary* tersebut dirancang secara generik, sehingga tidak dapat merepresentasikan data yang secara spesifik terdapat pada peraturan perundang-undangan di Indonesia. Sehingga dengan membuat *vocabulary* untuk struktur data dan peraturan perundang-undangan di Indonesia, diharapkan dapat menjadi kontribusi dalam mendukung pembuatan KG untuk dokumen peraturan perundang-undangan.

Lex2KG merupakan *framework* yang dikembangkan pada penelitian ini, dapat mengonversi dokumen menjadi KG secara otomatis. Belum ada pekerjaan terkait yang melakukan automasi konversi dokumen peraturan perundang-undangan menjadi KG secara otomatis. Oleh karena itu Lex2KG diharapkan dapat menjadi kontribusi yang mendukung pembuatan dan penggunaan KG.

1.2 Permasalahan

Berikut ini adalah rumusan permasalahan dari penelitian yang dilakukan:

- Bagaimana merancang suatu sistem untuk melakukan konversi otomatis PDF peraturan perundang-undangan menjadi KG?
- Bagaimana membuat ontologi untuk memperjelas *requirement* dan mempermudah konversi dokumen PDF menjadi KG?
- Bagaimana evaluasi use case terhadap kg peraturan perundang-undangan tersebut serta evaluasi konversi / populasi dalam skala besar?
- Bagaimana pemeliharaan KG dapat dilakukan apabila terdapat peraturan perundang-undangan baru?

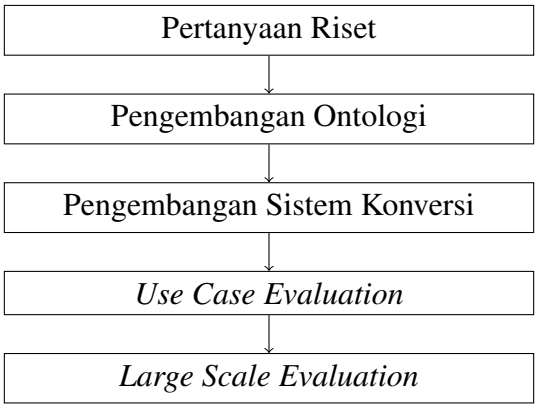
1.3 Batasan Permasalahan

Berikut ini adalah batasan dari penelitian yang dilakukan:

- Peraturan yang dipelihara kualitas konversinya dibatasi beberapa peraturan UU dan non-UU.
- Tidak dapat melakukan query terhadap KG yang kompleks (contoh: "Jika saya berusia 12 tahun, apakah saya boleh bekerja menurut UU Ketenagakerjaan")

1.4 Metodologi Penelitian

Metodologi penelitian dilakukan sesuai dengan yang diilustrasikan pada Gambar 1.3. Perancangan sebuah KG dimulai dengan memahami *competency questions* (*requirements*). *Competency questions* mencakup semua pertanyaan yang harus terdapat jawabannya didalam KG agar dapat digunakan pada *use case* yang akan disebutkan pada subbab selanjutnya. Setelah perntanyaan-pertanyaan sudah jelas, pertanyaan-pertanyaan tersebut dapat dikonversi menjadi SPARQL query untuk diperiksa apakah KG mengembalikan jawaban yang sesuai. Kemudian, dirancang ontologi (desain KG) yang dapat menjawab pertanyaan-pertanyaan tersebut. Perancangan ontologi ini mempermudah konstruksi KG.



Gambar 1.3: Metodologi Penelitian

Karena sulit untuk menjamin hasil konversi untuk semua PDF peraturan perundang-undangan, akan terdapat beberapa PDF yang akan dijamin konversinya benar yang selanjutnya akan disebut *maintained PDFs*. Pengembangan sistem konversi dilakukan secara iteratif. Setiap terdapat perubahan pada sistem, akan dicek apakah untuk semua *maintained PDFs* memberikan hasil konversi yang benar. Sistem konversi akan menerima input berupa PDF dan mengeluarkan output berupa KG dengan ontologi yang sudah dirancang pada tahap sebelumnya.

Untuk menjamin KG yang dibuat dapat menjawab *competency questions*, dalam penelitian ini dilakukan *use case evaluation* yaitu berupa demonstrasi contoh penerapan dari KG berdasarkan *competency questions*. Setelah semua *competency questions* terpenuhi, dilakukan *large scale evaluation* dimana PDF diluar *maintained PDFs* akan dikonversi kemudian dilakukan sampling pada hasil konversinya untuk dievaluasi secara kualitatif.

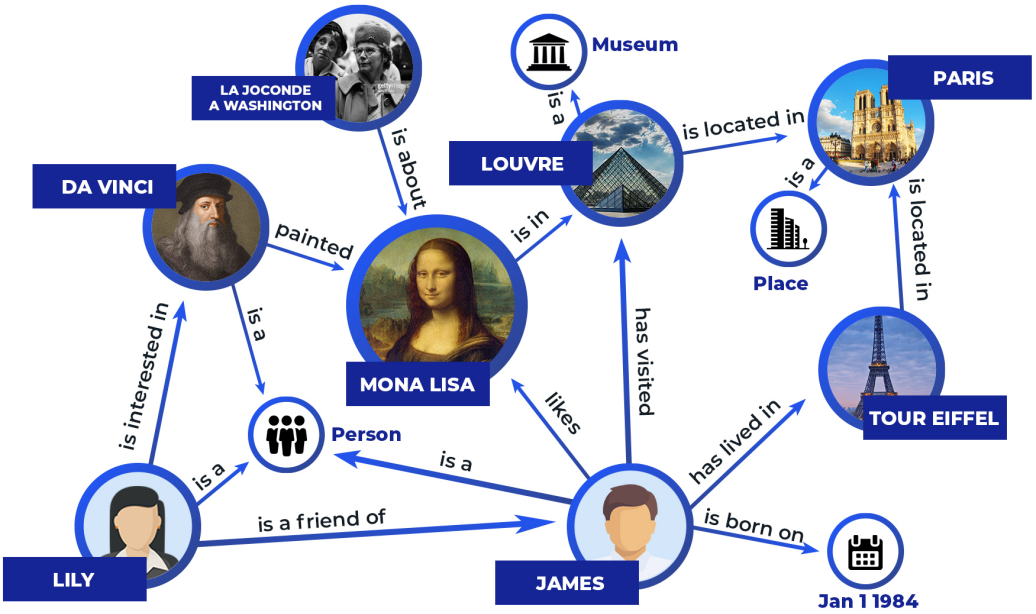
1.5 Sistematika Penulisan

Penelitian ini dibagi ke dalam 6 bab bahasan. Bab pertama menjelaskan mengenai pendahuluan yang berisikan latar belakang serta konteks penelitian yang dilakukan. Pada bab kedua dijelaskan mengenai tinjauan pustaka yang dilakukan dimana dijelaskan berbagai istilah dan bidang ilmu yang berkaitan dengan penelitian yang dilakukan. Lalu, pada bab ketiga dijelaskan mengenai perancangan penelitian yang dilakukan dalam menyusun penelitian ini. Selanjutnya, bab keempat akan membahas implementasi penelitian. Bab kelima akan membahas evaluasi dan analisis hasil penelitian. Bab keenam akan membahas kesimpulan penelitian dan saran untuk penelitan selanjutnya.

BAB 2
TINJAUAN PUSTAKA

2.1 Knowledge Graph

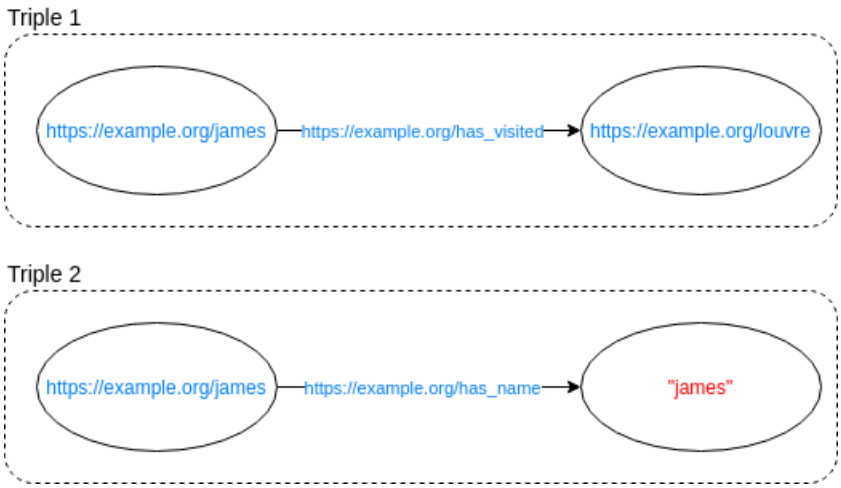
Knowledge graph merupakan model data berbasis *graph* yang menggambarkan entitas dunia nyata dan hubungan antara entitas-entitas tersebut (Seth, 2019). Sebagai contoh, pada gambar dibawah dapat dilihat terpadat entitas-entitas yang ditandai dengan bentuk lingkaran, dan hubungan antara entitas-entitas ditandai dengan bentuk anak panah. Dapat dilihat pada gambar dibawah bahwa terdapat entitas "James" dan "Louvre" dan properti "has visited" yang mengarah dari "James" ke "Louvre". Relasi tersebut dapat kita anggap sebagai sebuah pengetahuan yang dalam bahasa manusia dapat dituliskan sebagai "James pernah mengunjungi Louvre". Model data seperti ini cocok untuk menyimpan data dengan banyak jenis relasi antara entitas seperti pengetahuan umum, dibandingkan dengan *relational database* yang cocok untuk menyimpan data dengan relasi antar entitas yang sudah diketahui dan terbatas.



Gambar 2.1: Contoh *Knowledge Graph*. Sumber: `https://yashuseth.files.wordpress.com/2019/10/knowledge-graph.jpg`

2.2 Resource Description Framework

Resource Description Framework (RDF) merupkan model data yang memberikan pernyataan tentang suatu *resource* dalam bentuk subjek-properti-objek, atau sering disebut *triple*. Sebuah KG dapat direpresentasikan sebagai kumpulan *triple*. Subyek dan properti dalam *triple* direpresentasikan sebagai Uniform Resource Identifier (URI), sedangkan objek dapat direpresentasikan dengan URI atau *string literal*. Pada gambar dibawah, dapat dilihat terdapat KG dengan dua *triple*. Pada triple 1, subjek, properti, dan objek berturut-turut adalah `https://example.org/james`, `https://example.org/has_visited`, `https://example.org/louvre` yang mana semuanya berbentuk URI. Sedangkan pada triple 2, subjek, properti, dan objek berturut-turut adalah `https://example.org/james`, `https://example.org/has_name`, "james". `https://example.org/james`, `https://example.org/louvre` berturut-turut merupakan notasi untuk *resource* entitas James dan Louvre,e`https://example.org/has_visited` dan `https://example.org/has_name` merupakan properti yang menjelaskan hubungan antara subjek dan objek, dan "james" adalah *string literal* yang menunjukan nilai dari nama James.



Gambar 2.2: Contoh Triple

Terdapat beberapa format berkas dan sintaks untuk mengekspresikan RDF. Pada penelitian ini, penulis menggunakan format Terse RDF Triple Language (Turtle). Sebagai contoh, KG diatas dapat dituliskan dalam format Turtle seperti Kode 2.1.

```
<https://example.org/james> <https://example.org/has_visited>
↪  <https://example.org/louvre> .
<https://example.org/james> <https://example.org/has_name> "james" .
```

Kode 2.1: Contoh penulisan sebuah *triple* dalam turtle

Dengan fitur format Turtle, dua *triple* diatas dapat ditulis lebih singkat dengan mengganti prefix URI yang sama menjadi teks yang lebih singkat, dan menggabungkan penulisan beberapa *triple* dengan subjek yang sama sehingga subjek hanya perlu dituliskan sekali saja. Sebagai contoh, karena sebagaian besar URI memiliki prefix `https://example.org/`, kita dapat menggantinya dengan *string* `ex` dengan mendeklarasikan `@prefix ex: https://example.org/ .`. Kedua *triple* juga memiliki subjek yang sama yaitu `https://example.org/james`, oleh karena itu, penulisan kedua turtle bisa digabung dengan hanya menuliskan subjek sebanyak satu kali, kemudian menuliskan properti dan objek dipisah dengan tanda titik koma. Kode 2.2 adalah contoh penulisan dua *triple* diatas dalam bentuk yang lebih singkat dan lebih mudah dibaca oleh manusia.

```
@prefix ex: <https://example.org/> .

ex:james
  ex:has_visited ex:louvre ;
  ex:has_name "james" .
```

Kode 2.2: Contoh penulisan lebih singkat sebuah *triple* dalam turtle

2.3 SPARQL

SPARQL merupakan salahsatu RDF *query language* yaitu bahasa yang mampu mengambil atau mengubah data yang disimpan dalam format RDF. Sintaks SPARQL memiliki beberapa kesamaan dengan SQL, contohnya sebuah variabel diawali dengan tanda tanya seperti `?name`. Berikut adalah penjelasan sintaks SPARQL.

- `PREFIX`: Mendeklarasikan pemetaan prefix URI menjadi *string* yang lebih singkat. Hal ini dilakukan agar *query* lebih mudah dibaca oleh manusia seperti sintaks `@prefix` pada Turtle.
- `SELECT`: Menyatakan variabel yang akan ditampilkan sebagai output dari *query*.
- `WHERE`: Menyatakan kondisi *triple* yang akan ditampilkan pada output.

Berikut adalah contoh-contoh *query* SPARQL yang dilakukan pada contoh KG pada subbab sebelumnya.

2.3.1 Contoh Query 1

Kode 2.3 merupakan SPARQL query untuk menampilkan semua *triple* yang terdapat pada KG, akan memberikan output Tabel 2.1. Setiap baris yang terdapat pada output merupakan *triple* yang tedapat pada KG.

```
PREFIX ex: <https://example.org/>
SELECT ?s ?p ?o
WHERE { ?s ?p ?o }
```

Kode 2.3: Query 1

?s	?p	?o
ex:james	ex:has_visited	ex:louvre
ex:james	ex:has_name	"james"

Tabel 2.1: Output Query 1

2.3.2 Contoh Query 2

Kode 2.4 merupakan SPARQL query untuk menampilkan semua properti dan objek pada *triple* dengan subjek `https://example.org/james`. pada KG, akan memberikan output Tabel 2.2.

```
PREFIX ex: <https://example.org/>
SELECT ?properti ?objek
WHERE { ex:james ?properti ?objek }
```

Kode 2.4: Query 2

?properti	?objek
ex:has_visited	ex:louvre
ex:has_name	"james"

Tabel 2.2: Output Query 3

2.3.3 Contoh Query 3

Kode 2.4 merupakan SPARQL query untuk menampilkan semua objek pada *triple* dengan subjek `https://example.org/james` dan properti `https://example.org/has_visited`, dengan output Tabel 2.3. Artinya, *query* ini menampilkan semua tempat yang pernah dikunjungi james, yaitu louvre.

```
PREFIX ex: <https://example.org/>
SELECT ?visited_place
WHERE { ex:james ex:has_visited ?visited_place }
```

Kode 2.5: Query 3

?visited_place
ex:louvre

Tabel 2.3: Output Query 3

2.4 Peraturan Perundang-undangan

Menurut Undang-Undang Nomor 10 Tahun 2004 Pasal 7 Ayat 1, peraturan perundang-undangan adalah peraturan tertulis yang dibentuk oleh lembaga negara atau pejabat yang berwenang dan mengikat secara umum. Jenis dan hierarki peraturan perundang-undangan terdiri atas:

- Undang-Undang Dasar Negara Republik Indonesia Tahun 1945
- Undang-Undang/Peraturan Pemerintah Pengganti Undang-Undang
- Peraturan Pemerintah
- Peraturan Presiden
- Peraturan Daerah

Berikut adalah beberapa peranan perundang-undangan untuk Indonesia 1:

- Peraturan perundang-undangan merupakan kaidah hukum yang mudah dikenal (diidentifikasi), mudah diketemukan kembali, dan mudah ditelusuri. Sebagai kaidah hukum tertulis, bentuk, jenis,dan tempatnya jelas. Begitu pula pembuatnya.
- Peraturan perundang-undangan memberikan kepastian hukum yang lebih nyata karena kaidah-kaidahnya mudah diidentifikasi dan mudah diketemukan kembali.
- Struktur dan sistematika peraturan perundang-undangan lebih jelas sehingga memungkinkan untuk diperiksa kembali dan diuji baik segi-segi formal maupun materi muatannya.
- Pembentukan dan pengembangan peraturan perundang-undangan dapat direncanakan. Faktor ini sangat penting bagi negara-negara yang sedang membangun termasuk membangun sistem hukum baru yang sesuai dengan kebutuhan dan perkembangan masyarakat.

Menurut lampiran Undang-Undang Nomor 10 Tahun 2004, struktur peraturan perundang–undangan terdiri atas:

- Judul
- Pembukaan
- Batang Tubuh
- Penutup
- Penjelasan (jika diperlukan)
- Lampiran (jika diperlukan)

Bagian judul, pembukaan, dan penutup memuat metadata dari peraturan perundang-undangan tersebut seperti jenis, nomor, tahun pengundangan atau penetapan, nama, jabatan pembentuk, konsiderans, kasar Hukum, peraturan perundang-undangan. Konsiderans memuat uraian singkat mengenai pokok-pokok pikiran yang menjadi latar belakang dan alasan pembuatan peraturan perundang-undangan. Dasar Hukum memuat dasar kewenangan pembuatan peraturan perundang-undangan dan peraturan perundang-undangan yang memerintahkan pembuatan peraturan perundang-undangan tersebut. Oleh karena itu, kita dapat melihat keterkaitan antara peraturan perundang-undangan dengan melihat konsiderans dan dasar hukum dari peraturan tersebut.

Batang tubuh peraturan perundang-undangan memuat semua substansi peraturan perundang-undangan yang dirumuskan dalam pasal. Pengelompokkan materi peraturan perundang-undangan dapat disusun secara sistematis dalam buku, bab, bagian, dan paragraf atas dasar kesamaan materi. Pasal dapat dirinci ke dalam beberapa ayat. Pasal dan ayat dapat dibuat dalam bentuk kalimat maupun tabulasi rincian. Dalam bentuk tabulasi rincian, setiap rincian harus dapat dibaca sebagai satu rangkaian kesatuan dengan frase pembuka dan setiap rincian diawali dengan huruf (abjad) kecil dan diberi tanda baca titik. Suatu rincian dapat dibagi lagi ke unsur rincian yang lebih kecil.

Saat ini pembuatan dan pemanfaatan peraturan perundang-undangan masih dilakukan secara manual oleh manusia. Pembuatan peraturan perundang-undangan dilakukan dengan mengetikkan konten peraturan tersebut dengan format yang telah ditentukan. Metode pembuatan tersebut tidak masalah jika hanya akan dibaca oleh manusia karena manusia secara tidak sadar dapat melihat hirarki visual dari dokumen tersebut dan membuat data yang dilihatnya terstruktur di dalam otak. Metode ini membuat pemanfaatan peraturan perundang-undangan oleh mesin menjadi kurang efisien karena mesin perlu melakukan proses tambahan yaitu mengkonversi gambar menjadi data terstruktur.

2.5 *European Legislation Identifier* (ELI)

Negara-negara anggota EU memiliki cara mengatur informasi hukum yang berbeda-beda, sehingga cenderung menghalangi penemuan, pertukaran, dan penggunaan kembali informasi. European Legislation Identifier (ELI) adalah solusi yang bertujuan untuk mengatasi masalah-masalah tersebut. ELI merupakan sistem untuk membuat peraturan perundang-undangan tersedia secara daring dalam format terstandardisasi, sehingga dapat diakses dan digunakan oleh berbagai instansi. ELI bertujuan untuk untuk memfasilitasi akses, berbagi dan interkoneksi informasi hukum yang diterbitkan melalui nasional, Eropa dan sistem informasi hukum global. ELI dibangun berdasarkan persetujuan antara negara-negara EU.

ELI juga memiliki *technical implementation guide* [1] yang menjelaskan implementasi ELI dan spesifikasi yang tedapat pada ELI. Spesifikasi ELI antara lain adalah cara membuat URI suatu *resource*, *class*, dan properti. Ontologi yang digunakan pada ELI cenderung untuk menjelaskan metadata dari peraturan, dan tidak memiliki pemodelan khusus untuk menjelaskan *subdivision* dari peraturan. Nyatanya, ELI menggunakan class *eli:LegalResourceSubdivision* untuk mendefinisikan semua *subdivision*, tidak terdapat kelas khusus untuk variasi *subdivision* seperti bab, pasal, dan ayat. Selain itu, ELI dirancang untuk digunakan pada peraturan di Eropa, sehingga tidak memiliki ontologi untuk jenis peraturan di indonesia seperti "Peraturan Pemerintah".

2.6 Schema.org

Schema.org adalah aktivitas komunitas kolaboratif dengan misi untuk membuat, memelihara, dan mempromosikan skema untuk data terstruktur di Internet, di halaman web, dalam pesan email, dan lebih dari itu. *Vocabulary* Schema.org dapat digunakan dengan *encoding* yang berbeda, termasuk RDFa, Microdata dan JSON-LD. Banyak aplikasi dari Google, Microsoft, Pinterest, Yandex, dan lainnya sudah menggunakan *vocabulary* ini untuk memperkuat pengalaman yang kaya dan dapat diperluas. Schema.org memiliki *vocabulary* tersendiri untuk peraturan perundang-undangan [2]. Salah satu *vocabulary* nya adalah properti yang menjelaskan metadata suatu peraturan perundang-undangan seperti `legislationDate` dan `jurisdiction`.

[1] `https://publications.europa.eu/en/publication-detail/-/publication/8159b75d-5efc-11e8-ab9c-01aa75ed71a1`

[2] `https://schema.org/Legislation`

2.7 Wikidata

Wikidata adalah basis pengetahuan *free* dan terbuka yang dapat dibaca dan diedit oleh manusia dan mesin. Wikidata berperan sebagai penyimpanan pusat untuk data terstruktur dari proyek saudara Wikimedia termasuk Wikipedia, Wikivoyage, Wiktionary, Wikisource, dan lainnya. WikiProject adalah organisasi yang didirikan untuk mencapai tujuan penyuntingan tertentu, atau untuk mencapai tujuan yang berkaitan dengan bidang pengetahuan tertentu. Terdapat WikiProject tersendiri untuk bidang hukum dan peraturan perundang-undangan yaitu WikiProject Law [3].

[3] https://www.wikidata.org/wiki/Wikidata:WikiProject_Law

BAB 3
PERANCANGAN

3.1 *Competency Questions*

- Deskripsikan UU Cipta Kerja.
- Dapatkan semua pasal dari UU Cipta Kerja.
- Tampilkan teks dari UU Cipta Kerja Pasal 2 Ayat.
- Tampilkan semua pasal yang terdapat pada UU Cipta Kerja Bab 2.
- Dapatkan ayat yang mengandung teks "kompensasi" dan "buruh" yang disisipkan(*inserted*) oleh UU Cipta Kerja.
- Dapatkan komponen dari UU Cipta Kerja yang menyisipkan pasal ke UU Ketenagakerjaan dan tampilkan pula teks dari pasal tersebut.
- Dapatkan komponen dari UU Cipta Kerja yang mengubah pasal UU Ketenagakerjaan dan tambilkan teks sebelum dan setelah pengubahan.
- Dapatkan komponen dari UU Cipta Kerja yang menghapus pasal UU Ketenagakerjaan dan tampilkan pula teks dari pasal tersebut.
- Dapatkan komponen dari UU Cipta Kerja yang menghapus pasal UU Ketenagakerjaan dan tampilkan pula teks dari pasal tersebut.
- Tampilkan banyaknya penyisipan, pengubahan, dan penghapusan pasal yang dilakukan UU Cipta Kerja terhadap UU lain.
- Tujukan pasal pada UU Ketenagakerjaan setelah dilakukan *update* oleh UU Cipta Kerja.
- Tujukan pasal pada UU Cipta Kerja yang tidak melakukan *update*.
- Tujukan pasal pada UU Cipta Kerja yang menghapus pasal yang disahkan setelah tahun 2001.
- Tujukan pasal yang disisipkan oleh UU Cipta Kerja dan rujukan yang terdapat pada pasal tersebut jika ada.

- Tampilkan peraturan yang paling banyak di-*update* oleh UU Cipta Kerja.
- Tampilkan semua peraturan serta peraturan yang ditimbangnya.
- Tampilkan 10 peraturan dengan pasal terbanyak.
- Tampilkan peraturan yang pernah di-*update*.
- Tampilkan 10 tempat terbanyak peraturan disahkan.
- Tampilkan 10 peraturan paling banyak ditimbang.
- Tampilkan semua peraturan yang melakukan *update* dan banyaknya pasal yang di-*update* oleh pasal tersebut.
- Tampilkan jumlah triple tentang struktur peraturan.
- Tampilkan jumlah triple tentang metadata peraturan.
- Tampilkan jumlah triple tentang teks peraturan.
- Tampilkan jumlah triple tentang rujukan peraturan.
- Tampilkan jumlah triple tentang *update* peraturan.
- Tampilkan jumlah triple tentang *class assignment*.
- Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya.

3.2 Perancangan URI

Setiap entitas yang berbeda perlu diberikan URI yang berbeda. Penggabungan KG sangat umum dilakukan karena secara struktur dapat dilakukan dengan mudah, dan dapat memperkuat data dari masing-masing KG. KG memiliki entitas-entitas dengan URI yang unik, oleh karena itu saat menggabungkan sebuah KG A dan sebuah KG B, entitas yang berbeda harus memiliki URI yang berbeda. Sebuah masalah dapat terjadi apabila URI tidak dirancang dengan benar. Sebagai contoh, apabila terdapat entitas dari KG A dengan URI `https://example.org/trump` dengan maksud Donald Trump mantan presiden Amerika Serikat, dan entitas dari KG B dengan URI yang sama dengan maksud permainan kartu Trump, saat kedua KG ini digabung akan menjadi informasi yang berkontradiksi.

Pada penelitian ini, semua URI menggunakan prefix `https://example.org/lex2kg/` yang selanjutnya akan disebut *basePrefix*. Prefix ini dapat diganti dalam program agar dapat disesuaikan oleh pengguna program. Semua entitas dokumen dan komponennya menggunakan prefix URI

`https://example.org/lex2kg/peraturan` yang selanjutnya disebut *peraturanPrefix*, dan semua resource lainnya menggunakan `https://example.org/lex2kg/ontology` yang selanjutnya disebut *ontoPrefix*.

3.3 Perancangan URI Class

URI Class adalah URI yang menjelaskan *class* dari entitas tersebut. Semua URI Class mengikuti pola `{ontoPrefix}/{className}` dimana `className` adalah nama *class* tersebut. Berikut adalah semua *class* komponen beserta URI dan deskripsinya.

Nama Class	**Deskripsi**
`Peraturan`	Peraturan perundang-undangan
`DaftarBab`	Daftar satu atau lebih bab
`Bab`	Bab
`DaftarBagian`	Daftar satu atau lebih bagian
`Bagian`	Bagian
`DaftarParagraf`	Daftar satu atau lebih paragraf
`Paragraf`	Paragraf
`DaftarPasal`	Daftar satu atau lebih pasal
`Pasal`	Pasal
`VersiPasal`	Versi dari sebuah pasal. Sebuah pasal dapat memiliki satu atau lebih versi dari hasil amandemen
`DaftarAyat`	Daftar satu atau lebih ayat
`Ayat`	Ayat
`DaftarHuruf`	Daftar satu atau lebih huruf atau nomor
`Huruf`	Huruf atau nomor
`Menimbang`	Hal yang ditimbang oleh suatu peraturan
`Mengingat`	Hal yang diingat oleh suatu peraturan
`Segmen`	Segmen teks. Dapat memiliki rujukan ke suatu komponen peraturan

Tabel 3.1: Deskripsi Class Lex2KG

3.4 Perancangan URI Komponen Peraturan Perundang-undangan

URI peraturan perundang-undangan diawali oleh *peraturanPrefix*, kemudian diikuti oleh string yang mengidentifikasi peraturan tersebut. URI peraturan ini akan digunakan oleh entitas sebagai prefix yang selanjutnya akan disebut *docURI*. Berikut adalah jenis peraturan perundang-undangan yang diimplementasi pada penelitian ini beserta pola URI nya.

Variabel akan dikurung dengan kurung kurawal `{}`.

Jenis Peraturan	Pola URI	Penjelasan variabel
Undang-Undang Dasar 1945	`/uud`	
Undang-Undang	`/uu/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Pemerintah	`/pp/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Daerah Provinsi DKI Jakarta	`/perda_provinsi_dki_jakarta/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Gubernur DKI Jakarta	`/pergub_dki_jakarta/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Walikota Malang	`/perwali_malang/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan

Tabel 3.2: Konstruksi URI untuk Peraturan Perundang-undangan

Setiap komponen dokumen merupakan entitas, dan memiliki URI. URI sebuah komponen didahului oleh *docURI* dan semua komponen yang mengatasinya. Berikut adalah jenis komponen peraturan perundang-undangan beserta pola URI nya.

Class Komponen	Pola URI	Penjelasan variabel
`DaftarBab`	`{docURI}/bab`	`docURI`: URI peraturan yang mengatasi
`Bab`	`{daftarBabURI}/{no}`	`daftarBabURI`: URI `DaftarBab` yang mengatasi `no`: Nomor bab
`DaftarBagian`	`{babURI}/bagian`	`babURI`: URI `Bab` yang mengatasi
`Bagian`	`{daftarBagianURI}/{no}`	`daftarBagianURI`: URI `DaftarBagian` yang mengatasi `no`: Nomor bagian
`DaftarParagraf`	`{bagianURI}/paragraf`	`bagianURI`: URI `Bagian` yang mengatasi
`Paragraf`	`{daftarParagrafURI}/{no}`	`daftarParagrafURI`: URI `DaftarBagian` yang mengatasi `no`: Nomor paragraf

Class Komponen	Pola URI	Penjelasan variabel
DaftarPasal	{parentURI}/daftarpasal	parentURI: URI Bab, Bagian, Paragraf yang mengatasi
Pasal	{docURI}/pasal/{no}	docURI: URI peraturan yang mengatasi no: Nomor pasal
VersiPasal	{pasalURI}/versi/{date}	pasalURI: URI Pasal yang mengatasi date: Tanggal disahkan
DaftarAyat	{pasalURI}/ayat	pasalURI: URI Pasal yang mengatasi
Ayat	{daftarAyatURI}/{no}	daftarAyatURI: URI DaftarAyat yang mengatasi no: Nomor ayat
DaftarHuruf	{parentURI}/huruf	parentURI: URI Ayat, Huruf, VersiPasal, Menimbang, Mengingat, yang mengatasi
Huruf	{daftarHurufURI}/{id}	daftarHurufURI: URI DaftarHuruf yang mengatasi id: Nomor atau huruf
Menimbang	{docURI}/menimbang	docURI: URI peraturan yang mengatasi
Mengingat	{docURI}/mengingat	docURI: URI peraturan yang mengatasi
Segmen	{parentURI}/{name}	parentURI: URI Ayat, Huruf, DaftarHuruf, VersiPasal, Menimbang, Mengingat, yang mengatasi name: Nama teks

Tabel 3.3: Konstruksi URI untuk komponen Peraturan Perundang-undangan

3.5 Perancangan URI Properti

Properti ditandai dengan URI yang memiliki pola {ontoPrefix}/{name} dimana name adalah nama dari properti tersebut. Properti menghubungkan subjek dan objek. Subjek adalah berupa URI komponen, sedangkan objek dapat berupa URI komponen, string, *number* (bilangan), atau *date* (tanggal).

Nama	**Domain**	Objek	**Range**
nomor	Bab, Bagian, Paragraf, Pasal, Ayat, Huruf	string, number	Nomor atau huruf _identifier_ dari komponen
teks	Ayat, VersiPasal, Segmen	string	teks dari komponen
bab	DaftarBab	Bab	Bab
bagian	DaftarBagian	Bagian	Bagian
paragraf	DaftarParagraf	Paragraf	Paragraf
pasal	DaftarPasal	Pasal	Pasal
ayat	DaftarAyat	Ayat	Ayat
huruf	DaftarHuruf	Huruf	Huruf
segmen	Mengigat, Menimbang, Paragraf, VersiPasal, Huruf, DaftarHuruf	Segmen	Segmen
daftarBab	Peraturan	DaftarBab	DaftarBab
daftarBagian	Bab	Daftarbagian	DaftarBagian
daftarParagraf	Bagian	DaftarParagraf	DaftarParagraf
daftarPasal	Peraturan,Bab, Paragraf, Bagian	DaftarPasal	DaftarPasal
daftarAyat	Pasal	DaftarAyat	DaftarAyat
daftarHuruf	VersiPasal, Ayat, Huruf, Menimbang, Mengingat	DaftarHuruf	DaftarHuruf
merujuk	Segmen	entitas apapun	Teks merujuk suatu entitas
mengubah	Huruf	VersiPasal	Pengubahan pasal
menghapus	Huruf	VersiPasal	Penghapusan pasal
menyisipkan	Huruf	VersiPasal	Penyisipan pasal
tanggal	VersiPasal	date	Tanggal
jenisVersi	VersiPasal	'orisinal', 'penyisipan', 'pengubahan', 'penghapusan'	Jenis versi
versi	Pasal	VersiPasal	Versi suatu komponen

Nama	**Domain**	Objek	**Range**
`tentang`	`Peraturan`	string	Peraturan tentang
`menimbang`	`Peraturan`	`Menimbang`	Peraturan menimbang
`mengingat`	`Peraturan`	`Mengingat`	Peraturan mengingat
`disahkanPada`	`Peraturan`	date	Peraturan disahkan pada tanggal
`disahkanDi`	`Peraturan`	string	Peraturan disahkan di lokasi
`disahkanOleh`	`Peraturan`	string	Peraturan disahkan oleh
`jabatanPengesah`	`Peraturan`	string	Jabatan pengesah peraturan
`bagianDari`	komponen apapun	komponen apapun	Komponen subjek adalah bagian dari komponen objek

Tabel 3.4: Nama, domain, dan range properti Lex2KG

BAB 4
IMPLEMENTASI

4.1 OCR Ulang Berkas PDF

Dokumen peraturan perundang-undangan yang digunakan pada penelitian didapatkan dalam format berkas PDF. Penulis pada awalnya mencoba langsung mengekstraksi data dari PDF, tetapi penulis menemukan kesulitan. Salahsatu kesulitan yang penulis temui adalah terdapatnya salah pemindaian pada berkas PDF. Sebagai contoh, terdapat teks yang tertulis `Pasal` tetapi mengandung data `Pasai`. Selain itu, penulis juga menemukan bahwa setiap dokumen mengandung keslahan pemindaian yang bervariasi. Untuk suatu kasus, teks `(2)` selalu terdeteksi sebagai `(21` pada suatu dokumen, tetapi tidak pada dokumen lainnya. Pada kasus lainnya terdapat dokumen yang hampir tidak memiliki kesalahan pemindaian dan juga terdapat dokumen yang teksnya tidak dapat dibaca samasekali.

Walaupun dokumen-dokumen tersebut dipelihara oleh satu lembaga pada satu situs web, yaitu oleh Dewan Perwakilan Rakyat pada situs web dpr.go.id/jdih, masing-masing dokumen dibuat kedalam berkas PDF dengan cara yang berbeda-beda. Penulis tidak dapat mengetahui secara pasti metode apa yang digunakan, tetapi dari metadata yang didapatkan dari berkas PDF, penulis dapat membuat beberapa dugaan. Pada berkas PDF, terdapat metadata dengan nama **Creator**, dimana pada dokumen-dokumen peraturan Perudang-udangan yang didapatkan, tercantum nama-nama alat pencetak atau merk dari pencetak tersebut. Dari informasi tersebut penulis menduga bahwa terdapat sebagian peraturan perundang-undangan yang dibuat menjadi PDF dengan mencetaknya menjadi kertas terlebih dahulu kemudian di pindai oleh pemindai dan sebagian lainnya dikonversi langsung dari berkas *.docx*. Data yang dipindai adalah berupa gambar, artinya teks yang terdapat pada berkas PDF adalah hasil OCR (*optical character recognition*) oleh pemindai tersebut. Tabel 4.1 adalah salahsatu contoh dokumen beserta data yang tercantum sebagai **Creator** dan contoh kesalahan pemindaiannya yang banyak terjadi.

Dokumen	__Creator__	Kesalahan pemindaian
UU Nomor 13 Tahun 2003	ScanSoft PDF Create! 4	- (tidak ada)
UU Nomor 6 Tahun 2018	Canon	‘(2)‘ selalu dipindai ‘(21‘
PP Nomor 34 Tahun 2021	Fuji Xerox B9100	data teks tidak terbaca

Tabel 4.1: Metadata PDF Peraturan Perundang-undangan

Pemindaian berkas PDF dengan metode yang berbeda-beda memberikan kualitas hasil pemindaian yang berbeda-beda dan kesalahan pemindaian yang tidak konsisten. Untuk menyelesaikan masalah ini, penulis melakukan OCR ulang terhadap semua dokumen yang akan di konversi. Dengan melakukan OCR ulang menggunakan satu metode yang sama, penulis tidak hanya berhasil mendapatkan data hasil pemindaian berkas PDF dengan kualitas pemindaian yang konsisten untuk semua dokumen. Penulis memilih menggunakan Tesseract OCR 4 sebagai metode OCR karena sifatnya *open source* dan mendukung Bahasa Indonesia sebagai bahasa yang dipindai.

Untuk melakukan OCR ulang perlu dilakukan instalasi program yang dibutuhkan yang dapat dilakukan dengan *command* berikut pada sistem operasi Ubuntu 20.04. `ocrmypdf` merupakan *wrapper* untuk Tesseract OCR dan `tesseract-ocr-ind` adalah program tambahan untuk dapat melakukan OCR menjadi teks Bahasa Indonesia.

```
apt update apt install ocrmypdf tesseract-ocr-ind -y
```

Kode 4.1: Command untuk melakukan instalasi program OCR pada Linux OS

Setelah program diinstall, OCR dapat dilakukan dengan menjalankan *command* dibawah. Pada contoh dibawah dilakukan konversi `UU-2021-34.pdf` yang sudah ada menjadi `UU-2021-34_OCR.pdf` hasil OCR. Opsi `-force-ocr` diperlukan untuk melakukan OCR ulang pada berkas PDF yang sudah memiliki teks hasil OCR untuk di *overwrite*. Opsi `-jobs 4` menjalankan *command* dengan 4 *core* CPU, bilangan pada opsi ini dapat diganti sesuai kebutuhan. Opsi `-tesseract-config tesseract-config.cfg` menjalankan *command* dengan konfigurasi yang diberikan.

```
ocrmypdf -l ind --force-ocr --jobs 4 --tesseract-config tesseract-config.cfg
↪  UU-2021-34.pdf UU-2021-34_OCR.pdf
```

Kode 4.2: Command untuk menjalankan OCR

Konfigurasi Tesseract OCR yang digunakan adalah `tessedit_pageseg_mode 4` seperti yang terlihat pada isi file konfigurasi dibawah. `tessedit_pageseg_mode 4` digunakan agar Tesseract OCR mendeteksi teks sebagai *block*, bukan *word* atau *character*.

```
# tesseract-config.cfg
tessedit_pageseg_mode 4
```

Kode 4.3: Konfigurasi Tesseract OCR

4.2 Ekstraksi Berkas PDF menjadi Data *Span*

Agar dapat diproses oleh program, berkas PDF perlu diolah menjadi data berupa daftar teks dan posisinya yang selanjutnya akan disebut *span*. Sebuah *span* mengandung satu baris teks. Berikut adalah data yang dimiliki oleh sebuah *span*:

- `str`: teks yand dikandung
- `xL`: koordinat titik terkiri dari *span*
- `xR`: koordinat titik terkanan dari *span*
- `y`: koordinat titik teratas dari *span*
- `pageNum`: nomor halaman
- `id`: *identifier* unik untuk setiap span

Gambar 4.1 adalah contoh gambaran berkas PDF dan hasil ekstraksinya menjadi daftar *span*.

UNDANG-UNDANG REPUBLIK INDONESIA

NOMOR 13 TAHUN 2003

TENTANG

KETENAGAKERJAAN

Gambar 4.1: Contoh PDF yang akan diekstraksi

akan dipindai menjadi

```
- xL: 197.52
  xR: 442.79990000000004
  'y': 234.48000000000002
  str: UNDANG-UNDANG REPUBLIK INDONESIA
  pageNum: 1
  id: 0
- xL: 252.72
  xR: 387.6002
  'y': 255.12
  str: NOMOR 13 TAHUN 2003
  pageNum: 1
  id: 1
- xL: 291.12
  xR: 349.68048
  'y': 275.76
  str: TENTANG
  pageNum: 1
  id: 2
- xL: 257.28
  xR: 383.28015999999997
  'y': 296.4
  str: KETENAGAKERJAAN
  pageNum: 1
  id: 3
```


Kode 4.4: Hasil ekstraksi Gambar 4.1

4.3 Membersihkan *Noise* dari Halaman

Tidak jarang dokumen peraturan Perudang-undangan mengandung data *noise* yang tidak ingin kita ekstraksi seperti *header* dan *footer*. Pada penelitian ini, penulis menggunakan dokumen dari sumber yang sama sehingga memiliki format yang sama, dan juga posisi *header* dan *footer* yang hampir sama pada setiap dokumen.

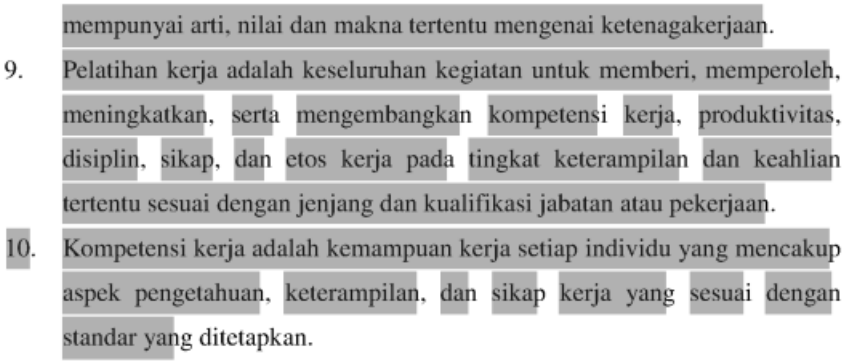
Gambar 4.2: Contoh header

Gambar diatas adalah contoh *header* yang terdapat pada setiap halaman. Dapat dilihat bahwa *header* selalu terdiri dari teks “PRESIDEN REPUBLIK INDONESIA” dan

diikuti oleh nomor halaman, dan selalu terletak di posisi yang hampir sama. Oleh karena itu, penulis memeriksa teks menggunakan *regex* dan posisi dari setiap *span*, kemudian menghapus *span* tersebut jika terdeteksi sebagai header.

4.4 Penggabungan Data Berkas PDF Asli dan Hasil OCR Ulang

Pada proses ekstraksi berkas PDF menjadi daftar *span*, penulis menemukan satu masalah yaitu data hasil OCR tidak konsisten dalam memindai angka. Seperti yang dapat dilihat pada gambar dibawah, angka 10 berhasil dipindai tetapi angka 9 tidak berhasil. Untuk kasus dibawah, angka hanya tidak terpindai pada hasil OCR ulang, tetapi terpindai dengan benar pada berkas PDF aslinya. Untuk menyelesaikan masalah tersebut, penulis melakukan ekstraksi data pada berkas PDF aslinya, kemudian menggabungkannya dengan data hasil pemindaian untuk melengkapi bagian yang tidak terpindai.


mempunyai arti, nilai dan makna tertentu mengenai ketenagakerjaan.

9. Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh, meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin, sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai dengan jenjang dan kualifikasi jabatan atau pekerjaan.

10. Kompetensi kerja adalah kemampuan kerja setiap individu yang mencakup aspek pengetahuan, keterampilan, dan sikap kerja yang sesuai dengan standar yang ditetapkan.

Gambar 4.3: Contoh nomor tidak terpindai

Penulis hanya menggabungkan data dua berkas PDF untuk kasus angka seperti yang disebutkan diatas. Hal tersebut dikarenakan pada umumnya kita dapat mentolerir kesalahan pemindaian pada teks, tetapi karena kegagalan pemindaian pada angka tersebut akan mempengaruhi struktur dokumen, penulis memutuskan solusi diatas. Sebagai contoh, jika nomor 8 berhasil dipindai dan nomor 9 tidak berhasil, maka struktur akhir yang dihasilkan akan mengandung semua isi nomor 9 didalam nomor 8, yang mana seharusnya adalah nomor yang terpisah. Berikut berturut-turut adalah contoh data yang dihasilkan jika nomor 9 tidak berhasil terpindai dan jika berhasil terpindai.

```yaml
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
    9. Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

Kode 4.5: Data jika nomor 9 tidak berhasil terpindai

```yaml
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
- type: point
  key: 9
  text: >-
    Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

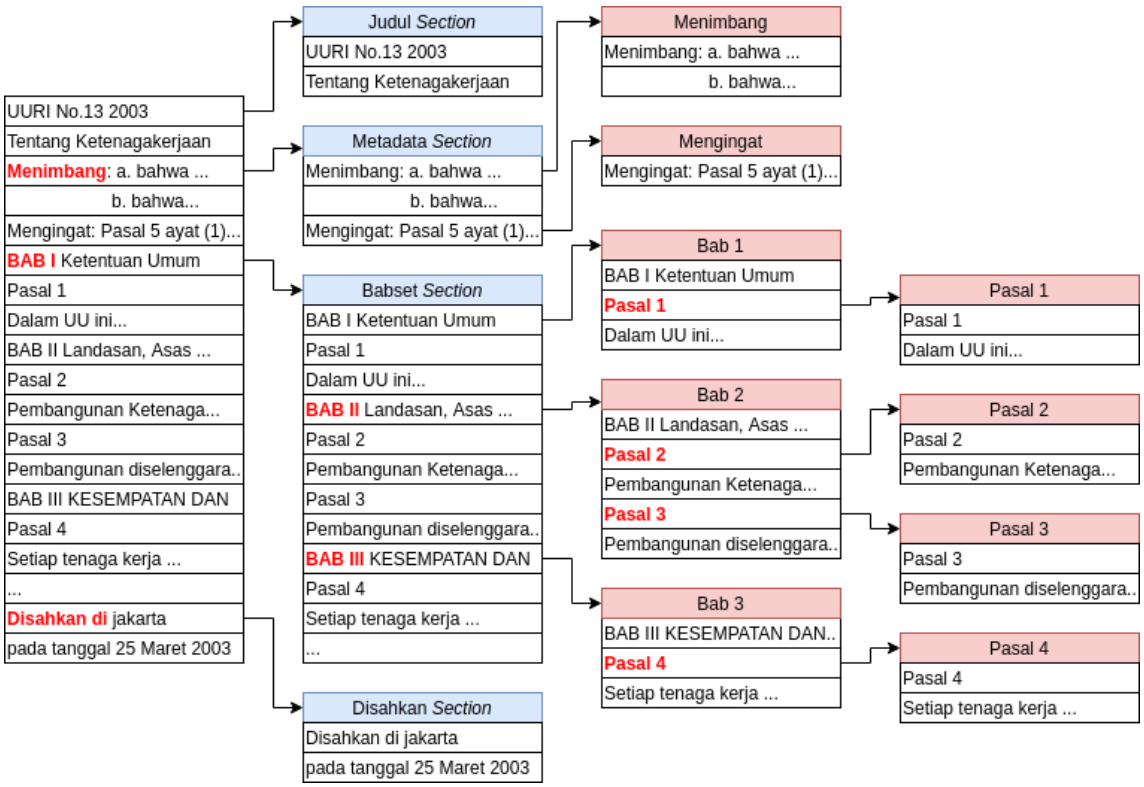
Kode 4.6: Data jika nomor 9 terpindai

4.5 Pengelompokan *span* menjadi Komponen

Teks yang membentuk suatu komponen terdiri dari satu atau lebih *span*, sehingga daftar *span* yang didapatkan dari hasil ekstraksi berkas PDF harus dikelompokkan sehingga setiap kelompok merepresentasikan *span* yang terdapat pada suatu komponen. Pengelompokan dilakukan dengan mendeteksi *span* awal dan *span* akhir dari sebuah komponen. Karena daftar *span* hasil ekstraksi bersifat 1 dimensi, hal ini bisa dilakukan dengan melakukan iterasi pada setiap *span*, kemudian menandai awal atau akhir dari sebuah kelompok apabila memenuhi suatu pola. Setelah ditandai, *span* akan dikelompokkan sebagai daftar *span* dari suatu komponen.

Pengelompokan tidak langsund dilakukan dari daftar *list* menjadi daftar komponen,

melainkan dibuat pengelompokan *section* untuk pembagian dokumen secara garis besar, kemudian baru dikelompokkan sebagai komponen dari masing-masing *section* tersebut. Pada gambar dibawah *section* ditandai dengan warna biru dan komponen ditandai dengan warna merah. Dapat dilihat bahwa daftar *span* data awal dikelompokkan menjadi beberapa *section* yaitu `judul`, `metadata`, `babset`, dan `disahkan`. Kemudian *span* pada section `babset` dikelompokkan menjadi komponen bab yaitu `Bab 1`, `Bab 2`, dan `Bab 3`. Kemudian *span* pada setiap komponen bab dikelompokkan menjadi komponen pasal. Pengelompokan bertingkat ini dilakukan agar fungsi untuk mendeteksi batas antara komponen tidak menjadi rumit.



Gambar 4.4: Ilustrasi Ekstraksi Daftar Span menjadi Data Struktur Komponen

Deteksi batas antara komponen atau *section* dilakukan dengan melihat data pada *span* seperti `str`, `xL`, `xR`, dan `y`. Pada gambar diatas, pola yang terdeteksi sebagai batas ditandai dengan huruf merah. Sebagai contoh, pada ekstraki paling kiri, dari daftar *span* asli menjadi *section*, dilakukan iterasi *span* dari atas dan akan dimasukkan ke dalam *section* `judul` sampai menemukan *section* yang diawali dengan kata **Menimbang**. Kemudian setiap *span* setelah span **Menimbang** tersebut akan dikelompokkan menjadi *section* `metadata` sampai menemukan section yang diawali kata **BAB I**.

4.5.1 Komponen Terurut

Beberapa komponen seperti `bab`, `bagian`, `paragraf`, `pasal`, `angka`, dan `huruf` memiliki sifat terurut. Dalam hal ini, yang termasuk terurut adalah:

- Komponen yang mengawali suatu komponen terurut akan selalu sama.
- Hanya terdapat satu komponen yang dapat mengikuti komponen terurut.

Contoh dari sifat pertama adalah `bab`, `bagian`, `paragraf`, `pasal`, `angka` pasti dimulai dari komponen nomor 1 seperti `bab 1` dan `pasal 1`, dan `huruf` pasti dimulai dari komponen `huruf a`. Contoh dari sifat kedua adalah hanya `pasal 2` yang dapat mengikuti `pasal 1` dan hanya `huruf b` yang dapat mengikuti `huruf a`. Dalam implementasi, penulis memastikan sifat-sifat ini dipatuhi, sehingga sebagai contoh apabila sebuah dokumen terdeteksi mengandung `bab 12` maka akan dijamin mengandung semua `bab 1` sampai `bab 11` dan apabila terdeteksi mengandung `pasal 196` maka akan dijamin mengandung semua pasal dari `pasal 1` sampai `pasal 195`.

4.5.2 Komponen Amandemen

Mengekstraksi *span* pada bagian amandemen merupakan salahsatu tantangan pada penelitian ini. Dibawah adalah gambar amandemen pada Pasal 17 UU Nomor 11 Tahun 2020 yang dilakukan terhadap Pasal 1 UU Nomor 26 Tahun 2007. Dapat dilihat bahwa kata `Pasal 17` dan `Pasal 1` yang digunakan sebagai pembatas antara komponen memiliki pola teks dan posisi yang hampir sama. Hal ini membuat pembedaan antara “Pasal biasa” dan “Pasal peng-amandemen” menjadi sulit dilakukan.

Pasal 17

Beberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

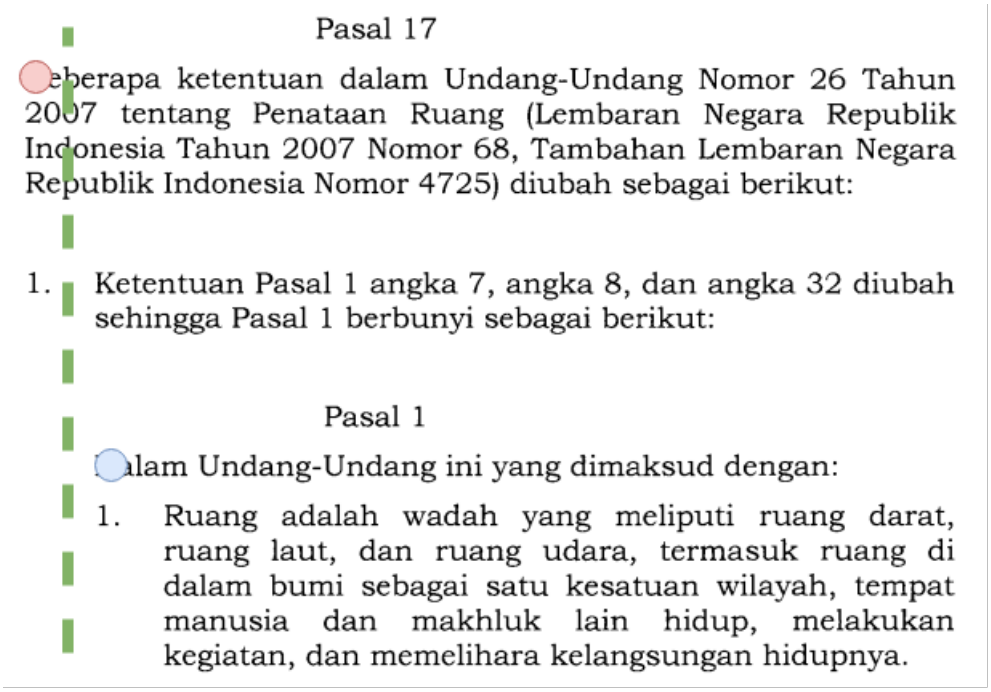
Pasal 1

Dalam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

Gambar 4.5: Contoh Amendemen pada UU Nomor 11 Tahun 2020

Untuk menyelesaikan masalah ini, penulis menggunakan koordinat xL *span* tepat setelah *span* yang bertuliskan "Pasal X". Pada contoh deteksi kompoonen amandemen di bawah, xL dari teks setelah pasal peng-amandemen (pasal biasa) ditandai dengan lingkaran merah, xL dari pasal yang diamandemen ditandai dengan lingkaran biru, dan garis hijau merepresentasikan batas pemeda xL antara pasal biasa dan pasal yang diamandemen. Pembedaan pasal biasa dan pasal amandemen hanya dilakukan jika dokumen memiliki komponen berupa amandemen.



Pasal 17

eberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

Pasal 1

alam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

Gambar 4.6: Deteksi Komponen Amandemen

4.6 Deteksi Sitasi

Setelah semua *span* dikelompokan menjadi suatu komponen, dilakukan deteksi sitasi pada teks pada setiap komponen untuk mengetahui apakah komponen tersebut menyebut komponen atau dokumen peraturan perundang-undangan lainnya. Berikut adalah daftar pola sitasi yang berhasil dideteksi pada penelitian ini.

Pola	**Contoh teks**	**URI Terdeteksi**
UUD 1945	Undang Undang Dasar Negara Republik Indonesia Tahun 1945	uud
Undang-Undang Nomor {x} Tahun {y}	Undang-Undang Nomor 26 Tahun 2007	`uu/2007/26`
ayat ({x})	ayat (1)	`uu/2003/13/pasal/169/ayat/1`
Pasal {x}	Pasal 156	`uu/2003/13/pasal/156`
Pasal {x} ayat ({y})	Pasal 156 ayat (2)	`uu/2003/13/pasal/156/ayat/2`

Pola	Contoh teks	URI Terdeteksi
huruf {x}, {y}, ... dan {z}	huruf a, b, c, d, dan e	uu/2003/13/menimbang/point/a uu/2003/13/menimbang/point/b uu/2003/13/menimbang/point/c uu/2003/13/menimbang/point/d uu/2003/13/menimbang/point/e
huruf {x} dan {y}	huruf a dan b	uu/2003/13/pasal/1/point/5/point/a uu/2003/13/pasal/1/point/5/point/b

Tabel 4.2: Long table caption.

Data sitasi mengandung data sebagai berikut:

- `start`: *index* karakter awal sitasi
- `end`: *index* karakter akhir sitasi
- `uri`: URI dokumen dan

```
textString: >-
  Pekerja/buruh yang diputus hubungan kerjanya berdasarkan alasan sebagaimana
  dimaksud dalam ayat (1), dapat memperoleh uang penggantian hak sebagaimana
  dimaksud dalam Pasal 156 ayat (4).
references:
  - start: 91
    end: 99
    uri: /uu/2003/13/pasal/158/ayat/1
  - start: 166
    end: 184
    uri: /uu/2003/13/pasal/156/ayat/4
```

Kode 4.7: Contoh teks pada Undang-Undang Nomor 13 Tahun 2003 Pasal 158 Ayat (4) beserta sitasi yang terdeteksi

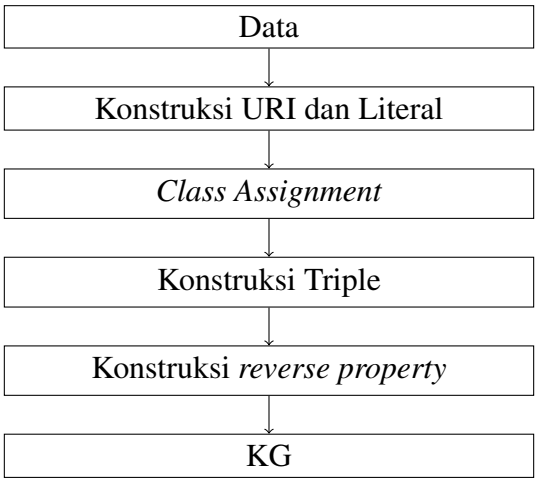
Untuk melakukan deteksi, teks akan melalui fungsi yang masing-masing memiliki pola untuk dideteksi. Oleh karena itu, untuk *substring* yang beririsan dapat memiliki lebih dari dua sitasi yang terdeteksi. Pada kasus tersebut akan diambil sitasi dengan *substring* terpanjang. Sebagai contoh, teks “Pasal 156 Ayat (4)” pada UU No.13 Tahun 2003 Pasal 158 Ayat (4) akan terdeteksi sebagai 3 URI sitasi sebagai berikut.

Teks	URI
Pasal 156 Ayat (4)	`/uu/2003/13/pasal/156/ayat/4`
Pasal 156	`/uu/2003/13/pasal/156`
Ayat (4)	`/uu/2003/13/pasal/158/ayat/4`

Tabel 4.3: Contoh teks dan URI yang terdeteksi dari teks tersebut

Pada kasus ini, yang akan diambil adalah teks "Pasal 156 ayat (4)" karena merupakan teks terpanjang.

4.7 Konstruksi KG



Gambar 4.7: Konstruksi KG

KG dikonstruksi dari data peraturan perundang-undangan yang sudah terdapat sebagai data terstruktur dalam memori. Setiap objek seperti peraturan dan komponen akan dibangun menjadi URI berdasarkan Tabel 3.2 dan Tabel 3.2. Literal seperti bilangan, *string*, dan tanggal juga dikonversi sesuai spesifikasi *literal type* pada turtle. Setiap objek juga akan di-*assign* ke masing-masing *class* sesuai yang terdapat pada Tabel 3.1. Kemudian triple dibangun berdasarkan relasi antara peraturan atau komponen dengan properti, domain, dan range sesuai spesifikasi pada Tabel 3.4.

4.8 Query

Query dilakukan terhadap KG untuk evaluasi *competency question* dan membuat demonstrasi untuk *use case evaluation*. Apache Jena Fuseki [1] digunakan sebagai SPARQL

[1] `https://jena.apache.org/documentation/fuseki2/`

server yang menyimpan KG dan melakukan query terhadap KG. Instalasi Fuseki dapat dilakukan dengan Kode 4.8.

```
wget https://downloads.apache.org/jena/binaries/apache-jena-fuseki-4.1.0.tar.gz
tar xvzf apache-jena-fuseki-4.1.0.tar.gz
cd apache-jena-fuseki-4.1.0
```

Kode 4.8: Command untuk melakukan instalasi Fuseki

Setelah instalasi Fuseki selesai dan berhasil masuk ke dalam folder Fuseki, Fuseki dapat dijalankan dengan KG dalam bentuk berkas turtle dengan Kode 4.9. Setelah command tersebut dijalankan, SPARQL query dapat dilakukan. Pada Kode 4.9, Fuseki dijalankan dengan endpoint */lex2kg*, sehingga SPARQL query dapat dilakukan menggunakan *client library* dengan endpoint `http://127.0.0.1:3030/lex2kg/sparql`, atau pada antarmuka yang tersedia pada `http://127.0.0.1:3030/dataset.html?tab=query&ds=/lex2kg`. Pada penelitian ini digunakan *client library* `fetch-sparql-endpoint` [2] untuk melakukan query dari bahasa pemrograman typescript.

```
java -jar fuseki-server.jar --file=lex2kg.ttl /lex2kg
```

Kode 4.9: Command untuk menjalankan Fuseki dengan berkas KG lex2kg.ttl

[2] `https://www.npmjs.com/package/fetch-sparql-endpoint`

BAB 5
EVALUASI

5.1 *Use Case Evaluation*

Use case dari KG yang dihasilkan dari Lex2KG dapat berupa 3 macam yaitu:

- SPARQL Query
- Visualisasi
- Question Answering Chatbot

5.1.1 SPARQL Query

Dengan menggunakan SPARQL server seperi Apache Jena Fuseki, KG yang dibuat dengan Lex2KG dapat dianalisis dengan menjalankan SPARQL query. SPARQL query memiliki fitur operasi *join*, *union*, *optional*, dan *negation pattern*. SPARQL juga menyediakan query analisis yang lengkap seperti *filter*, *aggregate*, dan *property path*. Dengan menggunakan SPARQL query, salah satu *competency question* "Tampilkan semua peraturan serta peraturan yang ditimbangnya." dapat dijawab dengan melakukan query Kode 5.1 yang akan memberikan output Tabel 5.1.

```
PREFIX o: <https://example.org/lex2kg/ontology/>
PREFIX uu: <https://example.org/lex2kg/uu/>

SELECT ?penimbang ?ditimbang
WHERE {
  ?penimbang o:menimbang ?menimbang .
  ?menimbangText o:bagianDari* ?menimbang .
  ?menimbangText o:merujuk ?ditimbang .
  ?ditimbang a o:Peraturan
}
ORDER BY DESC(?penimbang) DESC(?ditimbang)
LIMIT 10
```

Kode 5.1: SPARQL query untuk "Tampilkan semua peraturan serta peraturan yang ditimbangnya."

?penimbang	?ditimbang
uu:2020/8	uu:2018/12
uu:2020/6	uu:2020/2
uu:2020/6	uu:2015/1
uu:2020/6	uu:2014/1
uu:2020/2	uu:2020/1

Tabel 5.1: Output Query Kode 5.1

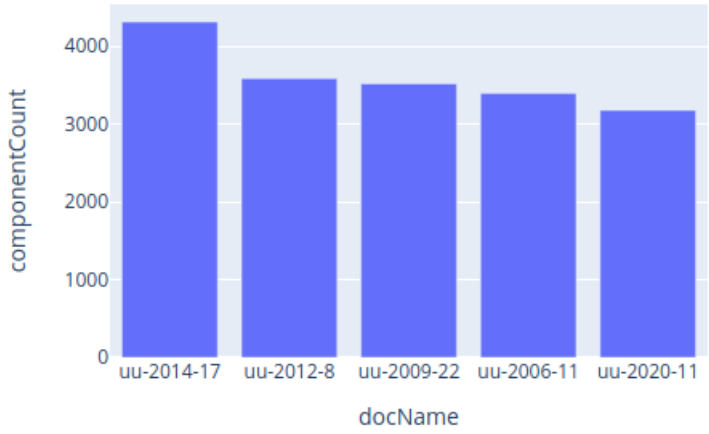
5.1.2 Visualisasi KG

Visualisasi meningkatkan visibilitas dan *explainability* KG. Pada KG dapat dilakukan visualisasi *node* dan *relation* antara *node*, atau visualisasi statistik dari KG tersebut. Kode 5.2 merupakan kode untuk melakukan visualisasi "Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya" menggunakan *visualization library* VizKG[1] dengan output Gambar 5.1.

```
import VizKG.visualize as VKG

sparql_query = """
PREFIX o: <https://example.org/lex2kg/ontology/>

SELECT ?docName (COUNT(?component) as ?componentCount)
WHERE {
  ?doc a o:Peraturan .
  ?component o:bagianDari* ?doc .
  BIND(REPLACE(REPLACE(STR(?doc), "https://example.org/lex2kg/", ""), "/", "-") as
↪  ?docName) .
}
GROUP BY ?docName
ORDER BY DESC(?componentCount)
LIMIT 5
"""

url = "http://127.0.0.1:3030/lex2kg/sparql"
bubble = VKG(sparql_query=sparql_query, sparql_service_url=url, chart='BarChart')
bubble.plot()
```

Kode 5.2: Visualisasi "Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya" menggunakan VizKG

[1] https://pypi.org/project/VizKG/



Gambar 5.1: Output VizKG untuk Kode 5.2

5.1.3 *Question Answering Chatbot*

```
Ada yang bisa dibantu?
1. Mencari Peraturan Perundang-undangan
1

Masukkan Keyword Peraturan: kerja

Berikut adalah peraturan yang judulnya mengandung kerja
/uu/2004/17
/uu/2016/3
/uu/1997/25
/uu/2016/15
/uu/2007/47
/uu/1991/1
/uu/2019/3
```

Kode 5.3: Skenario 1 Chatbot

5.2 *Large Scale Evaluation*

BAB 6
PENUTUP

Pada bab ini, Penulis akan memaparkan kesimpulan penelitian dan saran untuk penelitian berikutnya.

6.1 Kesimpulan

Berikut ini adalah kesimpulan terkait pekerjaan yang dilakukan dalam penelitian ini:

1. **Poin pertama**
   Penjelasan poin pertama.
2. **Poin kedua**
   Penjelasan poin kedua.

Tulis kalimat penutup di sini.

6.2 Saran

Berdasarkan hasil penelitian ini, berikut ini adalah saran untuk pengembangan penelitian berikutnya:

1. Saran 1.
2. Saran 2.

DAFTAR REFERENSI

Paulheim, H. (2016). Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 1-20.

Seth, Y. (2019, Oct). *Introduction to question answering over knowledge graphs.* Retrieved from https://yashuseth.blog/2019/10/08/introduction-question-answering-knowledge-graphs-kgqa/

LAMPIRAN

LAMPIRAN 1: CHANGELOG

lorem ipsum